PRUEBAS DE CAJA NEGRA

Técnicas de Prueba

El objetivo de las pruebas es descubrir el máximo número de fallos en el software, las pruebas añaden valor al producto (sistema de cómputo), al partir de un programa con errores y depurarlos. Las técnicas de prueba son el medio que ayuda a conseguir los objetivos de las pruebas a través de su ***sistematización.*** (Braude, 2012)

Para Braude, 2021, las técnicas de pruebas unitarias pueden tomar diferentes estrategias como Pruebas de caja negra y Pruebas de caja blanca.

En este apartado explicaremos la estrategia o prueba funcional CAJA NEGRA.

Pruebas de caja negra

Son pruebas funcionales que se llevan a cabo para saber si una aplicación o parte de ella proporciona la salida adecuada, en ellas se prueba que cumpla cada requerimiento al usar la entrada apropiada, se llaman así, porque no se pone atención a la aplicación (interior de la caja). Las pruebas de caja negra son parte de las pruebas funcionales dentro de las pruebas del sistema

Están enfocadas en los requerimientos funcionales del software, es decir, las técnicas de prueba de caja negra le permiten derivar conjuntos de condiciones de entrada que revisarán por completo todos los requerimientos funcionales para un programa. Como las pruebas de caja negra son ajenas a la estructura interna del objeto de las pruebas, permiten encontrar **errores** en:

a) Funciones incorrectas o faltantes
b) Errores de interfaz
c) Errores en las estructuras de datos o en el acceso a bases de datos externas
d) Errores de comportamiento o rendimiento
e) Errores de inicialización y terminación

En las pruebas de caja negra lo importante es el comportamiento que las entradas provocan en el mismo, con atención especial a los resultados producidos o salidas. Los contenidos no son visibles desde el exterior, estas pruebas se llevan a cabo en la interfaz del software. (Sánchez et al., 2011).

Cuando se generan **casos de prueba** de caja negra se pretenden demostrar que:

a) Las funciones del software son operativas.
b) La entrada se acepta de forma correcta.
c) Se produce una salida correcta.
d) La integridad de la información externa se mantiene.

Ventajas y Desventajas

Una ventaja de las pruebas de caja negra es que resulta sencillo ejecutarlas, ya que el encargado de la prueba se limita a introducir un conjunto de datos de entrada y a estudiar la salida. (Sánchez et al., 2011).

Y algunas desventajas de las pruebas de caja negra serían, el alto costo de mantenimiento de las pruebas automatizadas y el riesgo de dejar parte del código sin probar si no se diseñan correctamente. (Sánchez, et al. 2011).

Un problema importante en las pruebas de caja negra es la incertidumbre sobre la cobertura de las mismas, por lo tanto, es importante utilizar técnicas que permitan reducir y racionalizar el número de pruebas a realizar. Por lo tanto se identifica un conjunto finito de datos que será suficiente para realizar las pruebas cubriendo aceptablemente los distintos casos posibles.

Ejemplo

```
int CompararEnteros (int i, int j ) {
/* retorna 0 si i es igual a j, -1 si I es menos que
   J, y +1 si j es mayor que i */
if (i % 35 == 0 ) return -1;
if ( i == j ) return 0;
if (i < j )  return -1;
else return 1;
}
```

Se desea realizar pruebas sobre un módulo denominado CompararEnteros, el número de posibles combinaciones de enteros que podrían pasarse a esta función es elevadísimo, suponiendo enteros de 2 bytes se estaría hablando de todas las posibles parejas de valores entre -32,768 y +32,767. Por lo tanto se necesita encontrar un elemento único de cada clase que sea equivalente a los demás de su clase y que produciría salidas similares. De tal forma que identificaríamos tres clases de enteros: negativos, positivos y cero, para posteriormente probar solo 9 parejas: positivo-positivo, positivo–negativo, positivo–cero, cero–positivo, etc. por lo que el número de pruebas se reduce de más de 10^9 a solo 9.

Como podemos observar es necesario utilizar técnicas para la aplicación de las pruebas de caja negra, éstas pueden ser:

A. Particiones equivalentes
B. Análisis de valor de la frontera

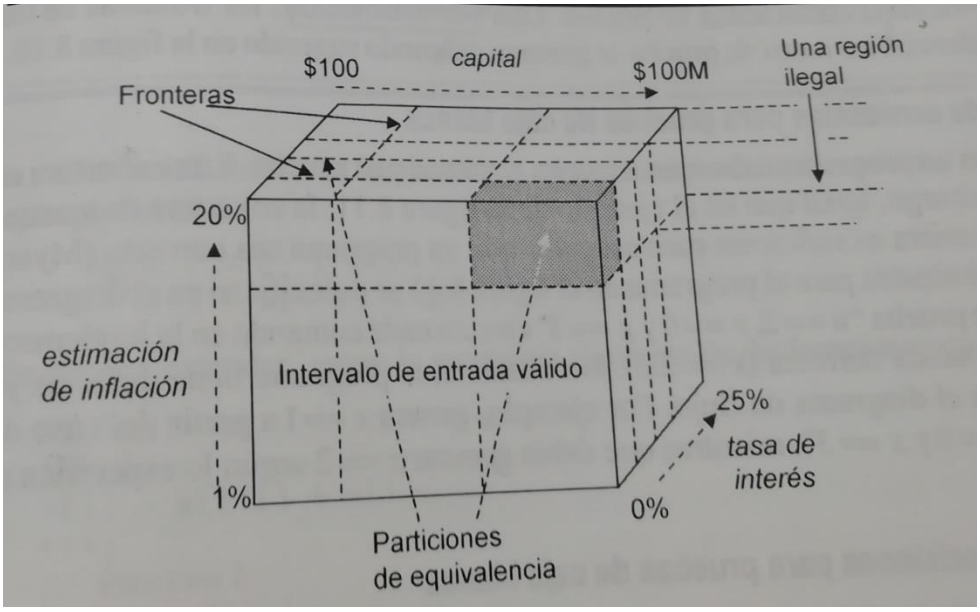
PARTICIONES EQUIVALENTES PARA PRUEBA DE CAJA NEGRA

Ya se estableció que no es posible probar todas las combinaciones de tal forma que se buscan casos de prueba representativos, por lo tanto surge la técnica de partición de equivalencia.

La partición de equivalencia es la división de los datos de entrada de las pruebas en subconjuntos tales que si cualquier entrada única tiene éxito, entonces es probable que todas las demás entradas tengan éxito. (Braude, 2011).

Ejemplo:

Se utilizan tres variables para una aplicación de finanzas: principal, tasa de interés y estimación de la inflación. El problema es representar de la mejor manera el conjunto infinito de posibilidades con un conjunto finito tan representativo como sea posible. (Braude, 2011).



La prueba de particiones equivalentes, también llamada de dominio, es una técnica basada en el modelo combinatorio y puede aplicarse a diferentes niveles, desde funciones y procedimientos de programación procedural, métodos de las clases bajo orientación a objetos. Por lo tanto se dice que una unidad es una pieza de código que recibe una serie de entradas y produce una serie de salidas, que puede incluir un valor de retorno, en caso de funciones. Las entradas pueden ser explícitas como parámetros o valores leídos de consola, pero también pueden ser implícitos, como valores tomados de estructuras de datos globales, de archivos o bases de datos. (Fernández, 2006).

De manera similar, las salidas puedes ser explícitas como los valores de regreso, escrituras a consola o impresora y parámetros pasados por nombre que sean modificados o implícitas a archivos y bases de datos o estructuras de datos globales.

Cada variable de entrada o salida tienen un dominio, es decir, pertenece a un conjunto de donde toma su valor. El dominio puede ser el conjunto de valores asociados con un tipo de datos. (Fernández, 2006).

En otro ejemplo:
Si se tiene una función de números reales que se evalúa de acuerdo con:

$$F(x) = \begin{cases} \frac{\sqrt{(x+5)}}{} & \text{si } x > 4 \\ 3x^3 & \text{si } x < -2 \quad \ldots\ldots\ldots \text{(1)} \\ 0 & \text{en otro caso} \end{cases}$$

Imagen tomada de apuntes de Fernández, 2006. Métodos combinatorios, recuperada el 2 marzo 2020 de: https://www.uv.mx/personal/jfernandez/files/2010/07/Cap2-Particiones.pdf

Se tendrán tres comportamientos diferentes: uno para valores negativos menores a -2, otro entre -2 y 4 y otro más para valores superiores a 4. En los últimos dos ejemplos el dominio se puede subdividir en varios subdominios, cada uno de los cuales será una clase de equivalencia respecto a la función o procedimiento que realiza la unidad. Otras clases de equivalencia se forman con los valores que no son permitidos y que son atrapados por las validaciones.

Para la función F(x) se tendrán tres clases

a) $(-\infty, -2)$
b) $[-2, 4]$
c) $(4, +\infty)$

variable	dominio	subdominios
Mes	{1, ... ,12}	(- ∞, 0), {2}, {4, 6, 9, 11}, {1, 3, 5, 7, 8, 10, 12} (12, +∞)
Día	{1, 2, ..., 31}	(- ∞, 0), {1, 2, ... , m} (m, +∞), m= 28, 29, 30 ó 31 según el caso
Año	[1900, 2199]	(- ∞, 1899], {a | a ∈ [1900, 2199] y a bisiesto} {a | a ∈ [1900, 2199] y a no es bisiesto}, [2200, +∞)

2. Casos de prueba

Entradas			Salidas esperadas		
día	mes	año	día	mes	año
-10	-2	1803	Datos fuera de intervalo		
-9	-3	1956	Datos fuera de intervalo		

Análisis de valor de la frontera para pruebas de caja negra

Análisis del valor frontera. Se llega a particiones de equivalencia mediante la investigación de los valores que limitan las variables dentro de la aplicación. (Braude, 2011).

Si se emplea el método de dominios cualquier valor dentro de una clase de equivalencia o subdominio, será igualmente para fines de prueba, sin embargo la experiencia indica que algunos valores son propensos a generar problemas. Un grupo de valores problemáticos son aquellos que se encuentra en los límites de un subdominio. (Fernández, 2006).

Surge así la técnica de valores a la frontera, donde para cada subdominio se eligen los casos de prueba como sigue:

a) El valor en la frontera
b) Un valor cercano a la frontera, pero dentro del subdominio
c) Un valor cercano a la frontera pero fuera del subdominio

ejemplo

En la misma sección se presentó la función F(x), con x real, que tiene tres subdominios:

a) $(-\infty, -2)$
b) $[-2, 4]$
c) $(4, +\infty)$

Aquí se tienen dos fronteras: -2 y 4, así que los casos de prueba serían:

Entrada	Salida esperada
-2.001	-24.036
-2.0	0
-1.999	0
3.999	0
4.0	0
4.001	2.080

Imagen tomada de apuntes de Fernández, 2006. Métodos combinatorios, recuperada el 2 marzo 2020 de:
https://www.uv.mx/personal/jfernandez/files/2010/07/Cap2-Particiones.pdf

CASOS DE PRUEBA

Los casos de prueba deben diseñarse a través de un plan de pruebas que incluya las siguientes recomendaciones:

1.- ¿Qué generó la prueba?
BRS (Business Requirements Specification)
2.- Contar con información que permita ubicar el error
a) Requerimiento funcional
b) Diagramas de clases
c) Descripción de clases

3.- Ubicar la Unidad (parte del código) donde se encuentra el error
4.- Generar la tabla de campos y dominios
5.- ¿Qué se va a probar?
6.- Ejecución de la prueba
7.- Resultado esperado
8.- Resultado obtenido (real)
9.- Análisis de resultados

Ejemplo

variable	dominio	subdominios
Mes	{1, ... ,12}	(- ∞, 0), {2}, {4, 6, 9, 11}, {1, 3, 5, 7, 8, 10, 12} (12, +∞)
Día	{1, 2, ..., 31}	(- ∞, 0), {1, 2, ... , m} (m, +∞), m= 28, 29, 30 ó 31 según el caso
Año	[1900, 2199]	(- ∞, 1899], {a | a ∈ [1900, 2199] y a bisiesto} {a | a ∈ [1900, 2199] y a no es bisiesto}, [2200, +∞)

2. Casos de prueba

Entradas			Salidas esperadas		
día	mes	año	día	mes	año
-10	-2	1803	Datos fuera de intervalo		
-9	-3	1956	Datos fuera de intervalo		
-11	4	2002	Datos fuera de		
15	6	2300	Datos fuera de intervalo		
1	2	1957	2	2	1957
12	2	1957	13	2	1957
28	2	1957	1	3	1957
29	2	1957	Datos fuera de intervalo		
28	2	2000	29	2	2000
29	2	2000	1	3	2000
1	4	1987	2	4	1987
17	4	1988	18	4	1988
30	4	1988	1	5	1988
31	4	1988	Datos fuera de intervalo		
1	8	1999	2	8	1999
20	8	1999	21	8	1999
30	8	1999	31	8	1999
31	8	1999	1	9	1999

Imagen tomada de apuntes de Fernández, 2006. Métodos combinatorios, recuperada el 2 marzo 2020 de:
https://www.uv.mx/personal/jfernandez/files/2010/07/Cap2-Particiones.pdf

Bibliografía

Braude, 2011, Ingeniería de software, una perspectiva orientada a objetos, séptima reimpresión Alfaomega Grupo Editor, S.A de C.V., México, ISBN 970-15-0851-3 , páginas 539.

Fernández, Juan, M., 2006, Métodos combinatorios, recupera el 2 marzo 2020 de: https://www.uv.mx/personal/jfernandez/files/2010/07/Cap2-Particiones.pdf

Sánchez, Salvador, Sicilia, Miguel Ángel y Rodríguez Daniel, 2012, Ingeniería de software. Un enfoque desde la guía SWEBOK, primera edición, Alfaomega Grupo Editor, S.A de C.V., México, ISBN 978-607-707-420-5, páginas 547.